Project: Solar Energy Generation in California

Modeling Report

1. Classification of the problem

1.1 Type of Machine Learning Problem

This project is a supervised learning problem that falls under the category of classification. Specifically, it is a binary classification problem where the goal is to predict the Solar Technoeconomic Intersection variable, which refers to areas with high or low solar potential or feasibility (modes: 'Within', 'Outside').

1.2 Task Relevance

Our task is closely related to the energy sector, supporting energy planning, renewable energy development, and spatial analysis. It is particularly valuable for utility companies, policy analysts, and researchers focused on renewable energy infrastructure.

1.3 Performance Metric

The primary performance metric chosen for this problem is the f1-score. This metric is selected because it balances precision and recall, making it ideal for imbalanced datasets.

Additional metrics used include precision and confusion matrix, as they provide deeper insights into model performance and help fine-tune the decision threshold.

ROC Curve

Another performance evaluation method we used is the Receiver Operating Characteristic (ROC) curve which is a graphical representation that shows the predictive performance of a binary classifier across all classification thresholds. It plots the True Positive Rate (TPR) against the False Positive Rate (FPR). A key metric derived from the ROC curve is the Area Under the Curve (AUC), which quantifies the classifier's performance. An AUC of 1.0 indicates perfect performance, while an AUC of 0.5 suggests that the model performs no better than random guessing. Although AUC alone doesn't capture all aspects of model performance (such as precision or recall), it is a good single-number summary of overall model performance.

2. Model choice and optimization

2.1 Algorithms Tried

We trained four classifier models: RandomForest, SVM (Support vector machine), XGBoost, and KNN (k nearest neighbors). To optimize hyperparameters in various machine learning algorithms, we employed GridSearchCV. This method trains the model with all possible combinations of the specified parameter grid and utilizes cross-validation to evaluate each combination. The optimal combination is selected based on a performance metric, which in our case is the f1-score.

2.2 Model Selection

We used Grid-search to optimize parameters for the best f1-score with a cross-validation parameter of 5.

To expand the scope of the optimization search, the Gridsearch procedure was repeated on three training datasets: a) an oversampled (OS) training dataset scaled using a min-max scaler, b) an oversampled training dataset scaled using a robust scaler, and c) an undersampled (US) training dataset scaled using a robust scaler.

2.3 Parameter Optimization

To determine the most effective model with optimized parameters, the following table presents the f1-score performances of various models.

f1-scores after Gridsearch		OS, min-max scaling	OS, robust scaling	US, robust scaling
RandomForest	Train	1.0000	0.9983	0.9794
	Test	0.9506	0.9390	0.9446
SVM	Train	0.8976	0.9250	0.8884
	Test	0.9261	0.9153	0.9282
XGBoost	Train	0.9997	0.9997	0.9674
	Test	0.9451	0.9299	0.9376
KNN	Train	1.0000	1.0000	1.0000
	Test	0.9335	0.9359	0.9341

The RandomForest algorithm, trained on oversampled and min-max scaled data, yields the highest f1-scores on test datasets. However, the table shows many train datasets with an f1-score of 1, indicating overfitting. To determine which model is more generalized, we calculate the ratio of the test f1-score to the train f1-score. These values are presented in the following table.

f1-scores after Gridsearch	OS, min-max scaling	OS, robust scaling	US, robust scaling
RandomForest	0.9506	0.9406	0.9645
SVM	1.0318	0.9895	1.0448
XGBoost	0.9454	0.9302	0.9692
KNN	0.9335	0.9359	0.9341

The above ratios simply show how well a model performs with the test dataset compared to the train dataset. Higher the ratio more generalized the model is. With this criteria SVM model has worked the best, yet we discard the model trained on oversampled-min-max-scaled and undersampled-robust-scaled train data due their low train f1-scores.

2.4 Conclusion

With above analysis, we choose SVM with parameters C = 10, gamma = auto, and kernel = rbf as the best classifier model.

3. Interpretation of results

3.1 Error Analysis

The classification report of the best model selected is:

```
Classification Report:
              precision    recall  f1-score   support

           0       0.76      0.91      0.83       322
           1       0.96      0.88      0.92       758

    accuracy                           0.89      1080
   macro avg       0.86      0.89      0.87      1080
weighted avg       0.90      0.89      0.89      1080
```

The confusion matrix for training data set:

	1	0
1	2831	201
0	250	2782

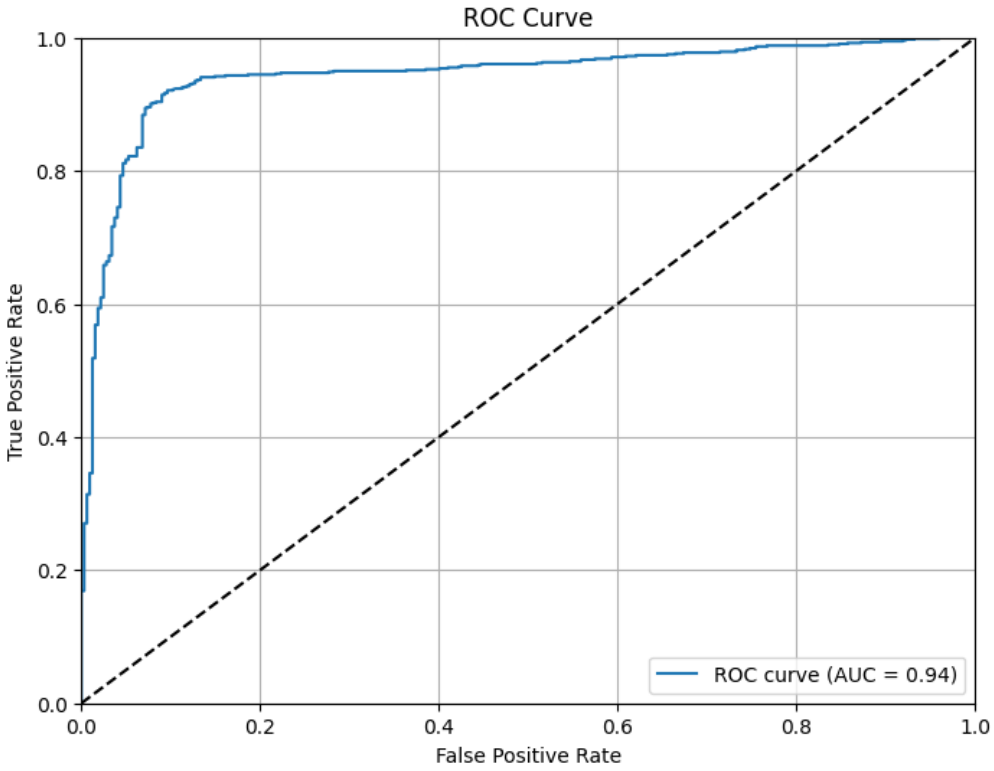
The confusion matrix for test data set:

	1	0
1	292	30
0	93	665

The following steps were taken to plot the ROC curve of the SVC (Support Vector Classification) model:

1. Predicted Probabilities: The predict_proba() method is used to get the predicted probabilities for the test set, specifically the probabilities for the positive class (class 1).
2. ROC Curve Calculation: The false positive rate (FPR) and true positive rate (TPR) are calculated using the roc_curve() function.

3. AUC Calculation: The ROC AUC score is computed using roc_auc_score() to measure the model's performance.



The resulting plot visualizes the trade-offs between sensitivity (TPR) and specificity (1 - FPR), providing insight into the model's classification performance. A diagonal dashed line representing random classification (AUC = 0.5) is also included for reference. With our resulting value of AUC = 0.94 we reached a strong predictive performance with our model which suggests that the model is very good at distinguishing between the two classes (positive/within the solar technoeconomic intersection and negative/outside of the solar technoeconomic intersection). The ROC curve also rises sharply towards the top-left corner which indicates that the model achieves high TPRs with low FPRs across a wide range of threshold settings.

3.2 Interpretability Techniques

After evaluating the models with GridSearchCV and selecting the best performing one, model interpretability is conducted using a key visual technique: SHAP (SHapley Additive exPlanations).

SHAP Summary Plot

SHAP (SHapley Additive exPlanations) is a model-agnostic technique used to interpret the predictions made by machine learning models. It assigns each feature an importance score,

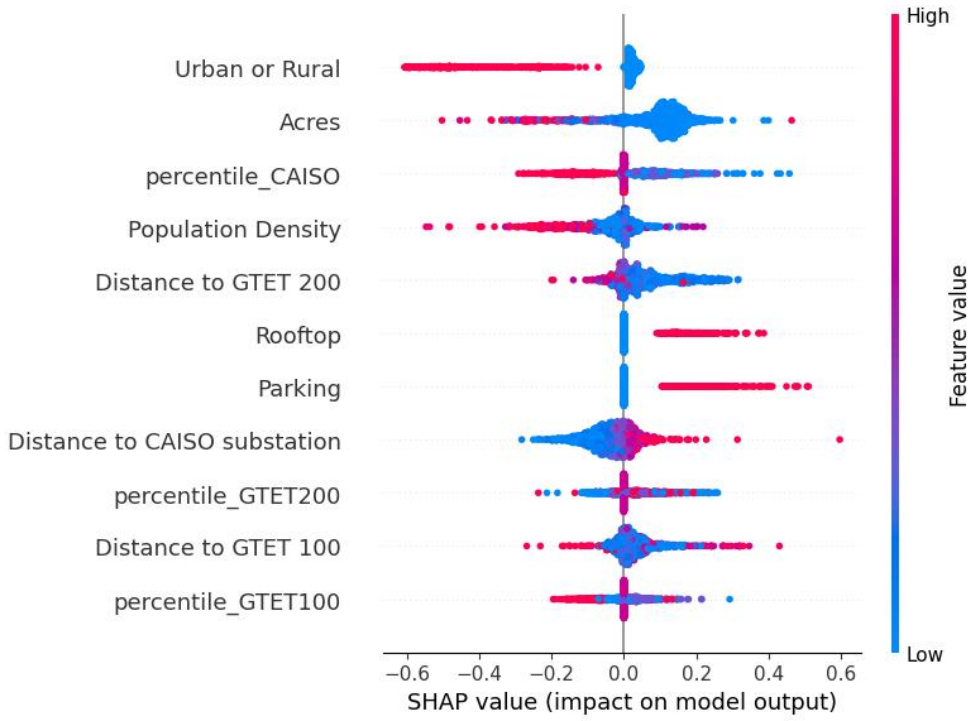
indicating how much each feature contributes to the model's output for a given prediction. This is particularly useful for understanding complex models.

We performed the following steps to generate a SHAP summary plot:

1. Background Data: To reduce computational cost, K-means clustering is used to select a smaller set of representative background samples (reduced to 10 samples in this case). This step helps speed up SHAP computation, as it reduces the size of the dataset that the SHAP algorithm needs to process.
2. SHAP Explainer: A KernelExplainer is initialized with the model's predict_proba() method and the background data. The explainer calculates SHAP values for each sample in the test set.
3. SHAP Values: The SHAP values are computed for the test set, which indicate the contribution of each feature to the prediction.
4. Summary Plot: The SHAP summary plot provides a global interpretation of the model. It shows the distribution of SHAP values for each feature, helping to understand the relative importance of each feature and the direction of its impact on the model's predictions. Features with higher SHAP values are considered more influential.

The resulting plot (see next section) helps in understanding the model's interpretability, showing which features are most important for predicting the target variable, and whether they have a positive or negative influence on the model's output.

3.3 Key Insights



Urban or Rural: This feature appears to be the most important predictor in the model. However, the concentration of red dots (high values - representing urban areas) on the *positive* SHAP value side suggests that being in an urban area *positively* impacts the model's output (higher probability of solar energy feasibility).

Acres: The second most important feature is "Acres." Here, the cluster of blue dots on the positive SHAP value side suggests that *lower* acreages tend to *positively* impact the model's prediction. This could indicate that smaller properties are more likely to be suited for solar energy.

percentile_CAISO: This feature has a distribution of both high and low values contributing negatively and positively to the SHAP value, depending on if the value is low or high. This feature has more impact to the model's output than the corresponding percentile_GTET100 or percentile_GTET200.

Population Density: Higher population density seems to have in particular an negative impact on the model's output. This means that, according to the SVC model, in areas with high population density, the probability of solar energy feasibility decreases.

Distance to GTET 200 ("GTET" refers to "Grid-Connected Transformer Equipment Terminal")**:** This feature has only a weak impact according to the SHAP values in the plot. However, this feature is on position 5 of the SHAP plot which means it has still an overall importance to the model output, e.g. by interacting with other features in a way that, on average, makes it more important than features lower down the list.

Rooftop and **Parking**: These features have a positive SHAP value. Rooftop suggests that properties with access to rooftops are more likely to be feasible for solar energy. Higher values lead to a positive impact on the model's output. Similarly, "Parking" has a positive influence.

Distance to CAISO substation: According to the SHAP values, this feature almost has no effect on the model output.

Distance to GTET 100: This feature shows some effect by showing an increase for the model output when this distance also increases. It's important to note that while "Distance to GTET 100" shows a trend, the magnitude of the SHAP values for this feature is relatively small compared to features like "Urban or Rural" or "Acres." Interestingly, the distances to GTET 200 or to CAISO substations have, on contrary to this feature, almost no effect according to SHAP.

percentile_GTET100 and percentile_GTET200: These percentile features seem to have both weak positive and negative impacts depending on their values. The spread of red and blue dots on both sides of the zero SHAP value line suggests that intermediate values can lead to higher probabilities, while very low and very high percentiles have the opposite impacts.

4. Evaluation of the model reliability

4.1 Bias and Fairness Considerations

For a bias free and a fair model performance, we resampled the train dataset. To that end, We employed random oversampling and cluster-centroid undersampling techniques. To ensure equal contributions from different variable, we scaled the training and test datasets using robust and min-max scalers. For model training, we use three sets of data: a) an oversampled (OS) training dataset scaled using a min-max scaler, b) an oversampled training dataset scaled using a robust scaler, and c) an undersampled (US) training dataset scaled using a robust scaler.

4.2 Robustness Testing

The model performances on train and test datasets were comparable, which ensured the robustness of the model. Hence, a detailed robustness test by adding random noise to the different variables was not carried out.

4.3 Deployment Readiness

The model is performing well, but as mentioned below we will conduct more tests before making it deployment ready. In our Preprocessing notebook, we experimented with a pipeline that currently only includes the scaling process. This pipeline could be further enhanced to incorporate the entire workflow, including data splitting, encoding, resampling, scaling, and prediction etc.

5. Optimization proposal

5.1 Potential Improvements

Additionally, we tried the implementation of a machine learning model using a stacking classifier, which combines multiple base models to improve predictive performance. In our case, the process includes hyperparameter optimization using Optuna and employs nested cross-validation to evaluate model performance and optimize hyperparameters. This also integrates various machine learning algorithms with a meta-learner to ensure improved performance. And finally, SHAP (SHapley Additive exPlanations) is used for model interpretability, allowing insights into the feature importance and model decisions.

The goal was to construct a robust and high-performing classification model capable of generalizing well to unseen data but since the process had its limitations (time and score) we decided for now not to implement this method for our modeling. Nevertheless, it is useful to explain how this approach works in the following because this approach could be pursued in the future to optimize our model further.

5.2 Stacking Classifier Overview

A stacking classifier is an ensemble learning method where predictions from multiple base models are combined using a meta-learner to make the final prediction. The base models used in this implementation are:

- Random Forest Classifier (RF)
- Support Vector Classifier (SVC)
- K-Nearest Neighbors (KNN)
- XGBoost (XGB)

The predictions from these base models serve as input to the meta-model, which in this case is a Logistic Regression model. We decided for a simple meta-model because it is best practice and eases the interpretability. The stacking approach leverages the strengths of different classifiers and aims to improve overall performance by combining them.

The use of base models that differ in their underlying algorithms (e.g., decision trees, support vector machines, and gradient boosting) increases the diversity of the ensemble, which often leads to better generalization compared to individual models.

5.2.1 Hyperparameter Tuning with Optuna within the Stacking Trial

To enhance the performance of the base models, Optuna is used for hyperparameter tuning. Optuna is a framework that automates the search for the best hyperparameters using a trial-based optimization method.

The inner loop of the nested cross-validation process focuses on tuning the hyperparameters for each base model. The models have key hyperparameters that can be optimized:

- Random Forest (RF): The number of estimators (n_estimators) is tuned.
- Support Vector Classifier (SVC): The regularization parameter (C) and the kernel coefficient (gamma) are adjusted.
- K-Nearest Neighbors (KNN): The number of neighbors (n_neighbors) and leaf size (leaf_size) are optimized.
- XGBoost (XGB): The number of boosting rounds (n_estimators) and the learning rate (learning_rate) are tuned.

Optuna's objective function evaluates the model's performance by fitting a stacking classifier with the suggested hyperparameters and computing the validation accuracy. The optimization process is guided by MedianPruner, which prunes trials that show poor performance early, thus accelerating the search. But in our case, it had its application limitations because our search was not exhausting enough (e.g. low number of trials selected) because of the high time consumption. But with changing parameters this method would be very useful so that we decided to keep it.

5.2.2 Nested Cross-Validation for Stacking Model Evaluation

To evaluate the performance of the stacking classifier and its base models, nested cross-validation is used. This method helps in assessing the model's ability to generalize, while ensuring that hyperparameter optimization does not lead to overfitting. The nested cross-validation consists of two primary loops here:

- Outer Loop: The outer loop splits the dataset into training and validation sets multiple times (using StratifiedKFold). This ensures that the model is tested on different subsets of the data, providing a robust estimate of generalization performance.
- Inner Loop: Inside each outer fold, the inner loop is responsible for performing hyperparameter optimization via Optuna. The model's performance is assessed on a validation set, and the best hyperparameters are selected.

Both loops use StratifiedKFold, which ensures that the class distribution is maintained across each fold, e.g. important for classification tasks with imbalanced datasets. The inner loop tunes the hyperparameters on the training set of each outer fold, while the outer loop evaluates the final model's performance on the validation set.

5.2.3 Stacking Model Training and its Final Evaluation

Once the hyperparameters are optimized in the inner loop, the stacking classifier is trained using the best hyperparameters on the entire training dataset (from each fold of the outer loop). After training, the model is evaluated on the validation set, and its performance metrics (accuracy and ROC-AUC score) are recorded.

To assess the final model's performance, it is retrained with the best hyperparameters across all outer folds and then tested on the hold-out test set. This final evaluation provides key metrics such as accuracy, ROC-AUC score, confusion matrix, and classification report, offering a comprehensive overview of the model's performance on unseen data.

5.2.4 Interpretability with SHAP

Model interpretability is a crucial aspect, especially in complex ensemble methods like stacking. We used again SHAP to provide interpretability by calculating the contribution of each feature to individual predictions.

The SHAP analysis starts by using k-means clustering to summarize the background data, reducing the computational complexity. We used k=1 to reduce the time amount to several minutes instead of hours. The KernelExplainer from SHAP is used to compute SHAP values for the test set, which represent the impact of each feature on the model's predictions. A summary plot is generated to visualize the distribution of SHAP values, highlighting the features that contribute most to the model's decision-making.

5.2.5 Conclusion of the Stacking Trial

The described stacking model is a powerful but complex machine learning approach combining the capabilities of several base classifiers through a stacking ensemble. By optimizing the hyperparameters of each model using Optuna and evaluating the performance through nested cross-validation, the model should achieve a higher generalization ability. With the use of SHAP we tried to further add transparency, allowing for a better understanding of how features influence predictions. However, the complexity and time consumption disadvantage lead to our final rejection of the stacking approach but it clearly stays as a possible way for potential optimization.